Tutorial IV.III - Objectives and Constraints in JuMP

Applied Optimization with Julia



Introduction

Welcome to this tutorial on objective functions and constraints in JuMP! In this lesson, we'll explore how to define objectives and add rules (constraints) to our optimization problems.

By the end of this tutorial, you'll be able to:

1. Define objective functions with variables in containers
2. Use containers (like arrays) to manage multiple similar constraints
3. Create more complex constraints based on conditions

Let's start by loading the necessary packages:

```
using JuMP, HiGHS
```

Now, let's create a model that we'll use throughout this tutorial:

```
another_model = Model(HiGHS.Optimizer)
println("Great! We've created a new optimization model.")
```

```
Great! We've created a new optimization model.
```

Section 1 - Objective Functions with Container Variables

Defining objective functions with variables in containers allows for scalable and dynamic model formulations. First, we need a container with variables for the objective function. For example:

```
@variable(modelName, variableName[1:3] >= 0)
```

Now, we can define an objective function with the container. For example:

```
@objective(modelName, Max, sum(variableName[i] for i in 1:3))
```

Here, every variable counts the same in the objective. If each variable contributes a different amount, we can multiply each variable by its coefficient:

```
coefficients = [2, 4, 3]
@objective(modelName, Max, sum(coefficients[i] * variableName[i] for i in 1:3))
```

Exercise 1.1 - Define arrays

Scenario: Imagine you're optimizing the production of 8 different products in a factory. Each product has a different profit margin per unit, and you want to maximize total profit. The profit margins are given as:

```
margin = [3, 2, 4, 5, 3, 6, 2, 4]
```

```
8-element Vector{Int64}:
 3
 2
 4
 5
 3
 6
 2
 4
```

Define an array of variables and an objective function for `another_model`. The variables should be called `production` and have a range from `1:8`, as they describe how many units of each product to make. They have a lower bound of `0`. The objective should be a Maximization of the total profit, which is the sum of `margin[i] * production[i]` over all products.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert length(production) == 8 && all(lower_bound(production[i]) == 0 for i in
1:8)
@assert typeof(objective_function(another_model)) == AffExpr
@assert all(coefficient(objective_function(another_model), production[i]) ==
margin[i]
    for i in 1:8) "Check your objective: each variable should be multiplied by
its margin."
println("Objective function with container variables defined successfully!")
```

Section 2 - Constraints within Containers

Defining constraints within containers allows for structured and easily manageable models. This is especially important when models become larger! To define a constraint within a container, we can do, for example, the following:

```
@constraint(modelName,
    constraintName[i in 1:3],
    variableName[i] <= 100
)
```

This would create a constraint called `constraintName` for each `i` - thus `1`,`2`, and `3` - where `variableName[1]`, `variableName[2]`, and `variableName[3]` are restricted to be maximally `100`.

> ⚠ **Warning**
> Right now, our factory model has an objective but no constraints. If you called `optimize!(another_model)` at this point, HiGHS would report the problem as unbounded: the production quantities could grow without limit, and so could the profit. The constraints you add next fix that.

Exercise 2.1 - Define constraints

Continuing our factory scenario: Each product has a maximum daily production capacity of `12` units due to machine limitations.

Define constraints called `maxCapacity` using an array of variables. The logic: Each production quantity defined in the previous task should be less than or equal to `12`.

```
# YOUR CODE BELOW


# Test your answer
@assert all(is_valid(another_model, maxCapacity[i]) for i in 1:8)
println("Constraints within containers defined successfully!")
```

Section 3 - Implementing Conditional Constraints

Conditional constraints are added to the model based on certain conditions, allowing for dynamic and flexible model formulations. To define a constraint within a container under conditions, we can do the following:

```
@constraint(modelName,
    constraintName[i in 1:3; i <= 2],
    variableName[i] <= 50
)
```

This would create a constraint called `constraintName` for each `i` - thus `1`,`2`, and `3` - where `variableName[1]`, `variableName[2]` are restricted to be maximally `50` and `variableName[3]` is not restricted.

Exercise 3.1 - Add a conditional constraint

Scenario extension: The first 4 products are new and have limited market demand.

Add a conditional constraint `limitedDemand` to the previous model. Condition: Only the first 4 production quantities have to be lower than or equal to `5`. Use the conditional syntax with `;` from the example above.

```
# YOUR CODE BELOW


# Test your answer
@assert limitedDemand isa Containers.SparseAxisArray "Please use the
conditional
    syntax `[i in 1:8; condition]` from this section to create the constraint."
@assert length(limitedDemand) == 4 "The constraint should only cover the first
4 products."
@assert all(is_valid(another_model, limitedDemand[i]) for i in 1:4)
println("Conditional constraint implemented successfully!")
println("Checking successful implementation.")
optimize!(another_model)
status = termination_status(another_model)
@assert status == MOI.OPTIMAL "Sorry, something didn't work out as the model
status is $status"
@assert objective_value(another_model) ≈ 250 atol=1e-4 "Although you have an
optimal solution,
    the objective should be 250 not $(objective_value(another_model)). Is the
model correct?"
println("Model components validated successfully!")
```

Visualization of Results

Let's visualize our optimal solution.

The next cell sets the look of the plots below: the course colors, quiet grey labels and no frame. Run it once. A plot picks it up when you add `course_style...` to its keywords, after a semicolon.

```
using Plots
# The course look for plots, used like this: bar(x, y; course_style...)
course_style = (
    palette=["#537E8F", "#F6B265", "#3D5369", "#A7C7C6", "#DB6B6B", "#FCCF9C"],
    background_color="#FFFDFC",
    fontfamily="Helvetica",
    titlefontfamily="Helvetica Bold",
    titlefontsize=18,
    titlefontcolor="#363D45",
    titlelocation=:left,
    foreground_color_text="#6B7480",
```

```
    foreground_color_guide="#6B7480",
    foreground_color_axis="#9AA1A8",
    foreground_color_border="#9AA1A8",
    yforeground_color_border="#FFFDFC", # no line on the left, the grid is
enough
    guidefontsize=12,
    tickfontsize=11,
    legendfontsize=11,
    foreground_color_legend=nothing,
    background_color_legend=nothing,
    grid=:y,
    gridcolor="#E8E4E0",
    gridalpha=1,
    framestyle=:axes,
    tick_direction=:none,
    yformatter=:plain,
    dpi=300,
    top_margin=4Plots.mm,
    left_margin=8Plots.mm,
    bottom_margin=6Plots.mm
)


# Assuming the model has been solved!!!
optimal_production = value.(production)

bar(1:8, optimal_production;
    title="Optimal production levels",
    xlabel="Product",
    ylabel="Production (units per day)",
    legend=false,
    course_style...,
    linecolor=course_style.background_color,
    xticks=1:8,
    ylims=:round, # bars start at 0, the axis ends on a round number
    size=(800, 400))
```

Take a close look at the chart: the bars show exactly two levels. Products 1 to 4 are produced at their demand limit of 5, and products 5 to 8 at their capacity limit of 12. Every variable sits at its tightest upper bound - what does that tell you about the constraints in this model?

Conclusion

Congratulations! You've completed the tutorial on advanced handling of objective functions and constraints in JuMP. You've learned how to define objective functions and constraints using container variables. Continue to the next file to learn more.

Solutions

You will likely find solutions to most exercises online. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.

Bibliography